

Brno University of Technology
Faculty of Information Technology

Image Processing (in English) (ZPOe 25/26)

Application of Basic Image Processing Filters on Human Face Images

Project Report

Adar Otieno `xotiena00`
Deniz Aksoy `xaksoyd00`
Eyobed Awel Nuri `xnuriey00`

May 10, 2026

Contents

1 Introduction **3**

2 Compilation and Running **3**

3 Implementations **3**

3.1 Common Input Images . . . 3

3.2 Eyobed's Filters . . . 3

3.2.1 Histogram Equalization . . . 3

3.2.2 Lens Distortion . . . 4

3.2.3 Posterize . . . 5

3.2.4 Vignette Blur . . . 5

3.2.5 Discussion . . . 6

3.3 Deniz's Filters . . . 6

3.3.1 Threshold Filter . . . 6

3.3.2 Diffuse Filter . . . 7

3.3.3 Crosshatch Filter . . . 7

3.3.4 Hue Rotate Filter . . . 7

3.3.5 Evaluation and Discussion . . . 8

3.4 Adar's Filters . . . 8

3.4.1 Python implementation . . . 8

3.4.2 Bloom Filter . . . 8

3.4.3 Orton Effect Filter . . . 9

3.4.4 Gamma Filter . . . 10

3.4.5 Dodge Filter . . . 11

4 Conclusion **12**

1 Introduction

The students have decided to rework the project by removing previously proposed trivial filters, namely blur, sharpen and emboss and replaced them with more sophisticated ones as per the suggestions of the professor. We developed 12 filters from different categories in total, 4 filters per student.

2 Compilation and Running

The project uses CMake for build management. The compilation and execution steps are shown below:

```
mkdir build && cd build
cmake .. && make
./project
```

The program expects input images in the `data/` directory. If you have extra images, put them there. The GUI has trackbars for image/filter selection and viewing windows to show the original and filtered images and an info panel for the description of the applied filter. If you cannot see the windows, drag them around to expose them.

3 Implementations

This section presents the filters developed by each student. Each student individually developed 4 filters, only the GUI is shared by the students.

3.1 Common Input Images

We used the following four images (Figure 1) for demonstrating the filters.



(a) Source 1 (b) Source 2 (c) Source 3 (d) Source 4

Figure 1: Original unprocessed input images

3.2 Eyobed's Filters

3.2.1 Histogram Equalization

Histogram Equalization is a contrast enhancement filter that redistributes the brightness values in an image. To prevent unwanted color shifting, we do it in the YCrCb color space only on the luminance channel then converting back to BGR.

The algorithm calculates the histogram frequencies, the Probability Density Function (PDF), and the Cumulative Distribution Function (CDF). The PDF for a pixel intensity i is calculated as:

$$PDF(i) = \frac{\text{count}(i)}{\text{total pixels}}$$

The CDF is then calculated as the cumulative sum of the PDF:

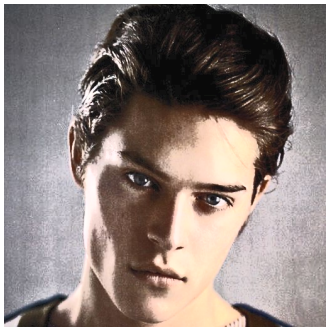
$$CDF(i) = \sum_{j=0}^{i} PDF(j)$$

Finally, the new pixel values are computed by scaling the CDF to the 0-255 range:

$$\text{Output}(x, y) = \text{round}(CDF(I(x, y)) \times 255)$$



(a) Result 1



(b) Result 2



(c) Result 3



(d) Result 4

Figure 2: Results after applying the Histogram Equalization filter.

3.2.2 Lens Distortion

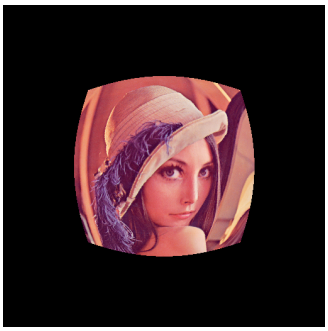
The filter uses an inverse geometric mapping to simulate lens distortion. For each output pixel (x, y), the source coordinate (s_x, s_y) is calculated through three steps listed below:

1. **Normalization:** Coordinates are centered and scaled by $R = \sqrt{c_x^2 + c_y^2}$ to maintain similar results for different image sizes:
$$d_x = \frac{x - c_x}{R}, \quad d_y = \frac{y - c_y}{R}$$

2. **Distortion:** A radial factor is computed using coefficient k [1]:
$$\text{dist} = 1 + k(d_x^2 + d_y^2)$$
Positive k produces barrel distortion, while negative k produces pincushion effects.

3. **Mapping:** Coordinates are scaled back to source pixel space:
$$s_x = d_x \cdot \text{dist} \cdot R + c_x, \quad s_y = d_y \cdot \text{dist} \cdot R + c_y$$

Pixels are then sampled using nearest-neighbor interpolation. The computational complexity is $O(W \times H)$.



(a) Result 1



(b) Result 2



(c) Result 3



(d) Result 4

Figure 3: Results after applying the Lens Distortion filter ($k = 5.0$).

3.2.3 Posterize

Reduces color depth by quantizing each channel into 8 discrete levels. A 256 value LUT is solved once to improve performance.

1. **Step Calculation:** The spacing between levels:

$$\text{step} = \frac{255}{L-1} \approx 36.4$$

2. **LUT Construction:** Each intensity i is rounded to the nearest level index, then scaled back:

$$\text{level}_i = \text{round}\left(\frac{i}{\text{step}}\right), \quad \text{LUT}[i] = \text{round}(\text{level}_i \times \text{step})$$

3. **Pixel Remapping:** Each BGR channel is independently mapped:

$$\text{Output}_c(x,y) = \text{LUT}[\text{Input}_c(x,y)], \quad c \in \{B, G, R\}$$

The complexity is $O(W \times H)$ for the pixel pass, plus $O(256)$ for LUT construction.



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 4: Results after applying the Posterize filter ($L = 8$ levels).

3.2.4 Vignette Blur

Softens edges while keeping the center sharp. it is 2 step. first box blur followed by a radial blend.

1. **Normalization:** Coordinates are centered and scaled by $R = \sqrt{c_x^2 + c_y^2}$ to maintain similar results for different image sizes:

$$d_x = \frac{x - c_x}{R}, \quad d_y = \frac{y - c_y}{R}$$

   The radial distance d is then computed as $d = \min(1.0, \sqrt{d_x^2 + d_y^2})$.

2. **Box Blur:** We do simple 15×15 mean kernel convolution on the interior pixels (7-pixel border excluded):

$$\text{Blur}(x,y) = \frac{1}{225} \sum_{dy=-7}^{7} \sum_{dx=-7}^{7} \text{Input}(x + dx,\ y + dy)$$

3. **Blending:** Mix original and blurred using d as the weight:

$$\text{Output}(x,y) = \text{Input}(x,y) \cdot (1 - d) + \text{Blur}(x,y) \cdot d$$

The complexity is $O(W \times H)$ since the blur and blend are computed in a single pass.



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 5: Results after applying the Vignette Blur filter using a 15x15 manual kernel.

3.2.5 Discussion

The author of this section studied the implementation of histogram equalization, lens distortion, posterization, and vignette blur filters. He believes that these filters are good demonstrations of concepts discussed in class. Additional techniques like look-up tables helped to take into consideration the performance aspect of image processing.

3.3 Deniz's Filters

Deniz's filters focus on fundamental image processing operations implemented through direct pixel-by-pixel matrix iteration.

3.3.1 Threshold Filter

The Threshold filter converts the image into a binary black-and-white representation using grayscale luminosity computation and threshold comparison. It is commonly used in OCR systems, segmentation, and preprocessing pipelines.

$$L = 0.299R + 0.587G + 0.114B$$

$$\text{Output} = \begin{cases} 255, & \text{if } L > T \\ 0, & \text{otherwise} \end{cases}$$

The algorithm processes each pixel independently with linear complexity:

$$O(W \times H)$$



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 6: Results after applying the Threshold filter.

3.3.2 Diffuse Filter

The Diffuse filter creates a frosted-glass effect by randomly replacing each pixel with a neighboring pixel within a selected radius.

$$(x', y') = (x + r_x,\ y + r_y)$$

The filter produces stochastic local distortions while maintaining linear computational complexity:

$$O(W \times H)$$



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 7: Results after applying the Diffuse filter.

3.3.3 Crosshatch Filter

The Crosshatch filter simulates sketch-style rendering using procedural diagonal line generation based on pixel coordinates.

$$(x + y) \bmod n = 0$$

Darker image regions generate denser hatch patterns, creating a pen-and-ink drawing effect.

$$O(W \times H)$$



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 8: Results after applying the Crosshatch filter.

3.3.4 Hue Rotate Filter

The Hue Rotate filter modifies image colors by rotating the Hue channel in HSV color space.

$$H' = (H + \Delta H) \bmod 180$$

The implemented version applies a 60° hue shift while preserving brightness and structural information.

$$O(W \times H)$$



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 9: Results after applying the Hue Rotate filter.

3.3.5 Evaluation and Discussion

The implemented filters demonstrate key image processing concepts through simple pixel-level operations. The Threshold filter supports segmentation and preprocessing, the Diffuse filter creates frosted-glass distortion, the Crosshatch filter produces sketch-like stylization, and the Hue Rotate filter changes color appearance while preserving image structure. Overall, the results show that lightweight algorithms can produce meaningful visual transformations with linear computational complexity.

3.4 Adar's Filters

Adar's contributions focus on creative glow and tone/brightness manipulation filters.

3.4.1 Python implementation

Reference Python sources for the filters are distributed as an archive on Google Drive: https://drive.google.com/file/d/19-CrN0iL8ds6OZ9fjDcawsgh3FoqknMW/view?usp=sharing. An interactive web demo of the implementations is available at http://161.97.70.30:7860/. The code was developed and tested with **Python 3.12**.

3.4.2 Bloom Filter

Core model

$$M(x) = \mathbf{1}_{I_g(x) \geq T}, \quad B = I \odot M, \quad G = G_\sigma * B, \quad I' = I + \alpha G$$

Notation T: threshold, σ: blur spread, α: glow strength, I': output.

Pixel-level transformation

$$I'_c(x,y) = I_c(x,y) + \alpha \sum_{i=-r}^{r} \sum_{j=-r}^{r} G_\sigma(i,j)\, B_c(x+i, y+j)$$

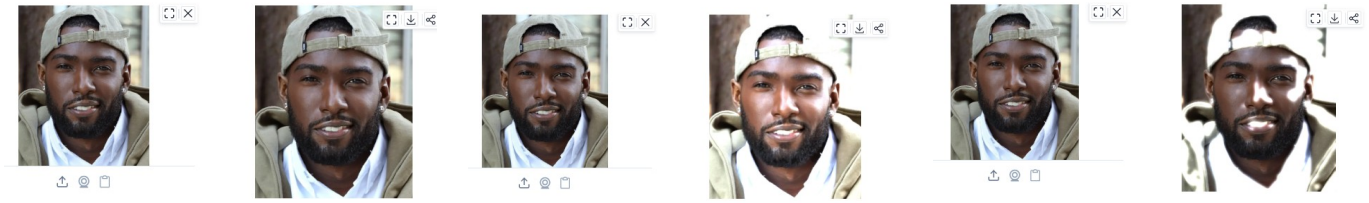
Only bright pixels feed the glow layer; high T can suppress bloom even with high α.

Algorithm (implementation)

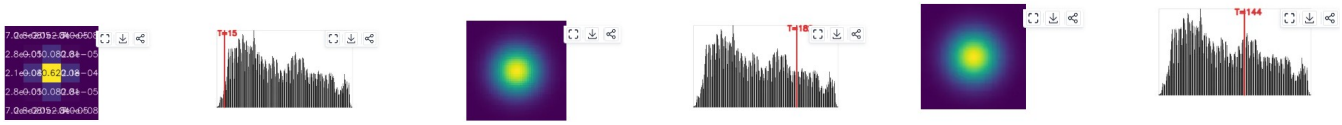
```
for each pixel (x,y):
  scan (2r+1)x(2r+1) neighborhood
  accumulate only neighbors with intensity > threshold
  glow = sum / ((2r+1)*(2r+1))
  out = clamp(src + strength * glow)
```

Implementation note Radius/threshold/strength are clamped in setter functions; output channels are clamped to $[0, 255]$.

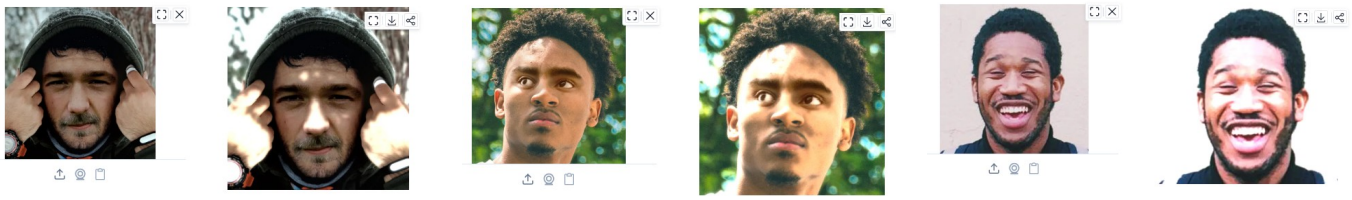
Experiment – Bloom



$T = 24,\ \alpha = 0.05,\ \sigma = 0.5$ $T = 21,\ \alpha = 0.65,\ \sigma = 1.2$ $T = 144,\ \alpha = 1.8,\ \sigma = 11.3$



Use cinematic highlight glow and light-source emphasis.



Compare stronger spatial glow than gamma/dodge. **Cost note** Per-pixel asymptotic cost is $O(k^2)$ because Bloom applies Gaussian blur on a $k \times k$ neighborhood.

3.4.3 Orton Effect Filter

Core model

$$S = 255 - \frac{(255 - I)(255 - B)}{255}, \quad B = G_\sigma * I, \quad I' = (1 - \alpha)I + \alpha S$$

Notation B: blurred image, S: screen blend, σ: blur spread, α: mix strength.

Pixel-level transformation

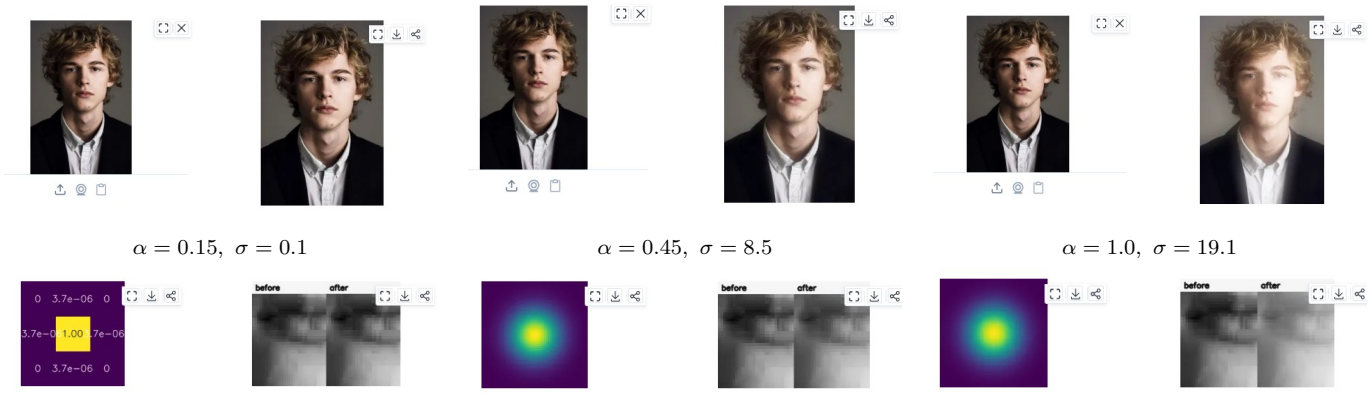
$$\begin{aligned} B_c(x, y) &= \sum_{i=-r}^{r} \sum_{j=-r}^{r} G_\sigma(i, j)\, I_c(x + i, y + j), \\ S_c(x, y) &= 255 - \frac{(255 - I_c(x, y))(255 - B_c(x, y))}{255}, \\ I'_c(x, y) &= (1 - \alpha) I_c(x, y) + \alpha S_c(x, y). \end{aligned}$$

Algorithm (implementation)

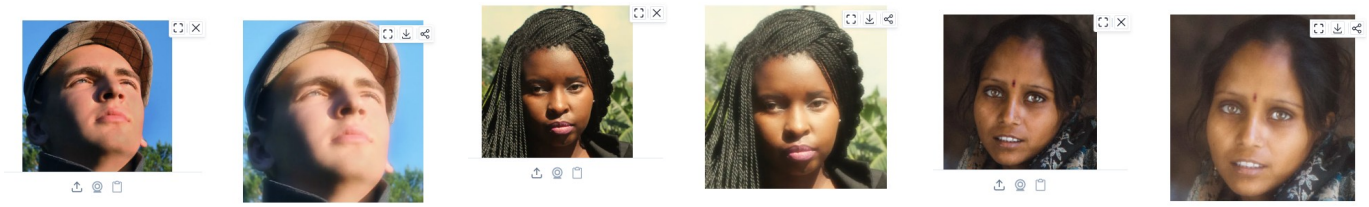
```
build 1D Gaussian kernel from sigma
horizontal pass with edge clamping
vertical pass with edge clamping
screen = 255 - ((255-src)*(255-blur)/255)
out = clamp((1-strength)*src + strength*screen)
```

Implementation note Implementation uses separable blur in `CV_64FC3`, then blends and clamps per channel.

Experiment – Orton



Use soft-focus portrait look with reduced micro-contrast.



Compare gentler and more diffuse than bloom. **Cost note** Per-pixel asymptotic cost is $O(k^2)$ due to Gaussian blur plus point-wise blend.

3.4.4 Gamma Filter

Core model

$$V_{\text{out}} = A\, V_{\text{in}}^{\gamma}, \qquad I' = 255 \left(\frac{I}{255} \right)^{1/\gamma}$$

Notation γ: tone-curve exponent, A: scale (usually 1), $I \in [0, 255]$: 8-bit input. **Why** 0..255 8-bit `uint8` images have 256 levels; divide by 255 to normalize to $[0, 1]$, then multiply by 255 to return.

Pixel-level transformation

$$I'_c(x, y) = 255 \left(\frac{I_c(x, y)}{255} \right)^{1/\gamma}$$

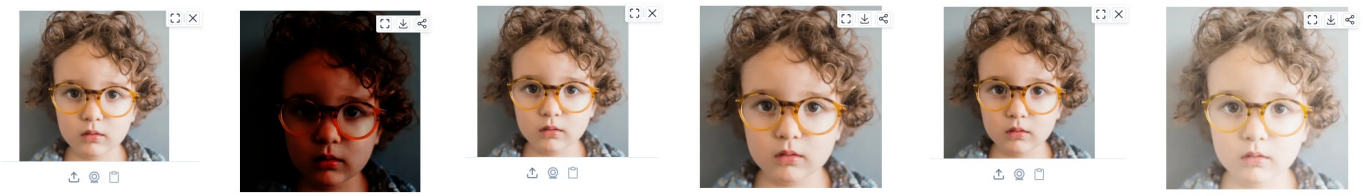
$\gamma < 1$: brighter midtones, $\gamma > 1$: darker midtones.

Algorithm (implementation)

```
invGamma = 1.0 / gamma
for each pixel/channel:
  normalized = value / 255.0
  corrected = round((normalized ^ invGamma) * 255)
  out = clamp(corrected)
```

Implementation note γ is clamped in the setter and correction is computed directly per channel (no LUT in C++).

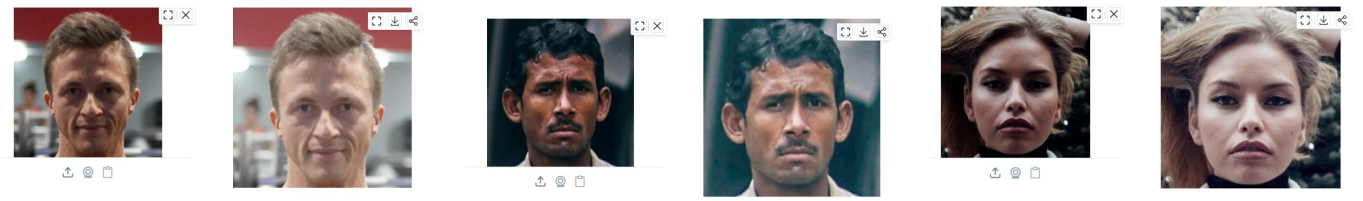
Experiment – Gamma



$\gamma = 0.2$ $\gamma = 1.0$ $\gamma = 1.95$



Use controlled global tone and midtone correction.



Compare smoother and less aggressive than dodge. **Cost note** Per-pixel asymptotic cost is $O(1)$ when implemented with LUT lookup.

3.4.5 Dodge Filter

Core model

$$I' \approx \frac{255\, I}{255 - sI}$$

Notation s: dodge strength, I: input intensity, I': output intensity.

Pixel-level transformation

$$I'_c(x,y) = \text{clip}\left(\frac{255\, I_c(x,y)}{\max(255 - s\, I_c(x,y), 1)}\right)$$

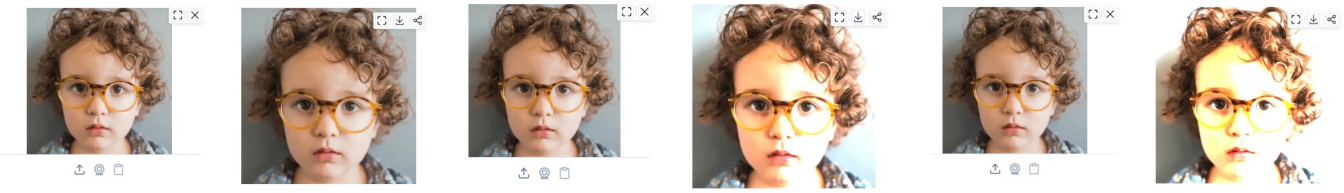
Higher s strongly boosts highlights and may clip to white.

Algorithm (implementation)

```
for each pixel/channel:
  dodge = (src*255) / max(1, 255 - src + protect)
  out = clamp((1-strength)*src + strength*dodge)
```

Implementation note Strength/protect are clamped in setters; denominator is guarded and output is clamped per channel.

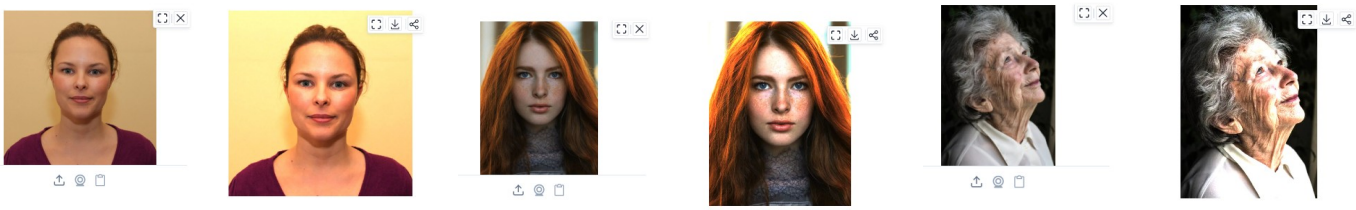
Experiment – Dodge



$s = 0.01$ $s = 0.5$ $s = 0.90$



Use stylized highlight brightening and sketch-like effects.



Compare more aggressive than gamma; non-spatial (no blur spread). **Cost note** Per-pixel asymptotic cost is $O(1)$ for direct pixel arithmetic.

4 Conclusion

This project demonstrates the application of fundamental image processing techniques. The students also estimated the computational complexity of each filter thanks to the knowledge they acquired in the course TINe. Moreover, we also learned the importance of using appropriate data structures and algorithms as well as performance considerations for image processing.

References

[1] Ma, L., Chen, Y., & Moore, K. L. (2003). *A New Analytical Radial Distortion Model for Camera Calibration*. arXiv preprint cs/0307046.